

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

InSPECtor: Improving SLEIGH Specification Veracity via Proxy

Michael Chesser, *Adelaide University;* Paul Quirk, *Defence Science and Technology Group;* Douglas Cooke and Guy Farrelly, *Adelaide University;* Surya Nepal, *CSIRO;* Damith C. Ranasinghe, *Adelaide University*

https://www.usenix.org/conference/usenixsecurity26/presentation/chesser

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

InSPECtor: Improving SLEIGH Processor Specification Veracity via Proxy

Michael Chesser[1], Paul Quirk[2], Douglas Cooke[1], Guy Farrelly[1], Surya Nepal[3], and Damith C. Ranasinghe[1]

[1]*Adelaide University*
[2]*Defence Science and Technology Group*
[3]*CSIRO*

Abstract

Processor specifications underpin critical security and program-analysis tools such as disassemblers, decompilers, and emulators, yet, their correctness is rarely examined. Errors in specifications distort program behavior, obscure vulnerabilities, and enable analysis-evasion techniques. Validating processor specifications is a non-trivial task. Our study is a significant undertaking to enable the systematic validation of open-source SLEIGH language specifications, predominantly used by Ghidra. We design and implement a testing framework based on an automated oracle validation strategy by *proxy*. Our approach leverages the structure encoded in a specification itself to enumerate decodable instruction forms and generate targeted initial states. Then differentially test the successful decoding and emulation of those instructions by comparing emulators exercising the processor specification against hardware references.

Applying InSPECtor across diverse, open-source specifications—x86-64, AArch64, ARM/Thumb, RISC-V, MSP430—embedding differences in specification styles, author preferences, and instruction set architecture designs, we uncovered over 38,920 discrepancies that led to 38,900 unique bugs with proposed fixes, identifying decoding and semantic defects as well as cross-vendor inconsistencies. We distill our findings into 8 concrete recommendations to drive future improvements. Our work underscores the importance of specification correctness and provides a practical tool to substantially improve the fidelity of SLEIGH processor specifications, strengthening the reliability of downstream security and analysis tools.

1 Introduction

SLEIGH is a domain-specific language (DSL) for describing processor specifications that decode and translate instructions into the architecture-agnostic intermediate representation (IR), Pcode [36]. There are over 35 processor families expressed in SLEIGH specifications, covering a diverse range of processors from PIC microcontrollers to modern x86-64 and AArch64 CPUs. Significantly, SLEIGH specifications underpin Ghidra [35], the dominant open source multi-architecture toolchain for reverse engineering and binary analysis, and multi-architecture emulators such as Alligator from Airbus [10], Icicle [8], MetaEmu [7] and Styx [22]. In Ghidra's disassembler and decompiler, SLEIGH specifications directly determine how binary code is decoded, lifted, and interpreted by analysts and automated scripts. As a result, SLEIGH specifications sit on the critical path of security workflows such as vulnerability discovery, malware analysis, fuzzing, firmware auditing, and program understanding.

This widespread reliance amplifies the consequences of specification errors: flaws in SLEIGH can propagate across multiple tools, users, and workflows. Yet, despite this central role, the reliability of SLEIGH specifications remains an underexplored problem.

In practice, SLEIGH specifications are large, painstakingly constructed artifacts that encode both instruction formats and their semantics. Developing faithful processor specifications is a non-trivial task. For instance, the ARM manual is 12,000 pages where complexities such as the operand-dependent behavior of instruction prevents a simple approach based on examining mnemonics. Additionally, the 2025 edition of the Intel Software Developer's Manual [1] exceeds 5,000 pages. The specifications must capture subtle architectural behaviors across diverse and often poorly documented instruction set architectures. Further, to address the demand for rich, fast, and safe computation, modern architectures are constantly evolving, with new instructions and features added each year[1], translating to an increase in instructions and complex instruction-bit encodings in ISAs. The painstaking construction and update of specifications to integrate architectural changes while maintaining backward compatibility makes the

[1]For instance, microprocessor advancements, such as Single Instruction, Multiple Data (SIMD), Arm Neon, Advanced Matrix Extensions (AMX), Advanced Vector Extensions (AVX) or cryptography extensions are often incorporated to cater to artificial intelligence, high-performance computing and security-focused workloads.

specifications intrinsically susceptible to bugs. Even small mistakes—such as missing side effects or incorrect semantic ordering—can propagate silently through static and dynamic analysis tools. These errors may lead to misleading decompilation output and unsound program analyses, ultimately undermining the conclusions drawn by security practitioners; often exploited by malware to detect virtual environments [6] and hide from analysis [29, 32, 33].

Importantly, the critical dependency on SLEIGH specifications has not been met by a commensurate, open source assurance of its correctness. Therefore, we are motivated to improve the correctness of SLEIGH processor specifications.

By enhancing the veracity of SLEIGH processor specifications, we expect to drive improvements across all their applications.

The Study. In this *first* study, we focus our efforts on improving the veracity of the SLEIGH processor specifications with the hope of generalizing our learning outcomes to broader specification assurance research. Importantly, these specifications need to be updated and reassessed for correctness as ISAs evolve and new instructions are added. Our efforts will facilitate testing implementations thoroughly, and automating this process in the future.

Previously, studies have examined improving the correctness of CPU emulators [23–25, 37], hypervisors [3], processor hardware [17, 31, 34], disassembly veracity (textual representation accuracy) [18, 28], while a few have tested the semantic correctness of binary lifters [9, 19] but specification-level correctness has received less attention. Although the work in [26] considered usability of Pcode for formal analysis and the authors in [4] made significant strides to validate the veracity of Sail specification language described semantic models for significant parts of ARMv8-A, RISC-V, and CHERI-MIPS, the correctness of the extensive multi-architecture support within the SLEIGH specifications has not been examined.

In seeking to design an approach to uncover specification bugs, we observe that a processor specification itself encodes enough structure to drive systematic validation. So, we introduce InSPECtor, a framework that automatically traverses SLEIGH specifications to enumerate decodable instructions, derive targeted execution states from semantic definitions, and perform differential testing against hardware reference systems. This enables high-coverage, specification-driven testing without redundant instruction exploration.

Our Contributions. In summary:

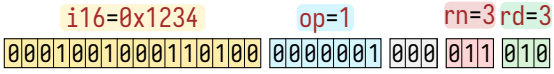
- We propose symbolically traversing SLEIGH specification decoding rules, a new, systematic methodology that aims to exhaustively generate instruction encodings, extracting edge cases for testing and deriving initial input states that trigger these edge cases.
- We design, implement, and evaluate the methodology in the prototype InSPECtor system for testing specifications, including, GenSYS—a test case generation tool that integrates these techniques and JuxtaPlayer—a robust differential testing environment capable of executing test cases on both emulators and hardware-assisted virtual machines.
- We analyze the discovered bugs, identify their root causes, propose fixes, and distill 8 recommendations for improving the SLEIGH DSL and preventing future issues.

Importantly, we use InSPECtor to test x86-64, AArch64, ARM/Thumb, RISC-V, and MSP430 SLEIGH specifications. Our work uncovered over 38,920 discrepancies. Manually triaging, categorizing, and prioritizing the discrepancies, we identified the root causes of the discrepancies leading to the discovery of 125 unique bugs with proposed fixes submitted to Ghidra maintainers. We open-source InSPECtor on GitHub: https://github.com/Sleigh-InSPECtor.

2 Instruction Encodings and SLEIGH Specifications Primer

Before describing our specification-driven testing approach, we briefly review ISA-level instruction encodings and the structure of SLEIGH specifications. Throughout this discussion, we use examples from a simplified ISA with a variable length instruction set, similar to x86-64, defined by the SLEIGH specification in Figure 1[2].

Instructions are represented as sequences of bytes, referred to as *instruction encodings*. Although each concrete instruction has a unique binary representation, instruction encodings within an ISA typically share a common structure. Bits are grouped together to form fields that have specific meanings, such as the registers, immediate values, and addressing modes. For example, in the simplified ISA, `addi r2,r3,0x1234` is encoded as follows:



SLEIGH specifications exploit this structure by expressing decoding as a hierarchy of declarative rules. These rules, called *constructors*, are organized into groups called tables. The SLEIGH specification in Figure 1 is annotated with the key components of each constructor:

- **Pattern section**: defines the mapping from an instruction encoding to the other components of the specification via Boolean expressions that reference the instruction encoding and the current decoder *context*—a SLEIGH-specific mechanism to encode or transfer information between constructors which we will discuss later.

[2]For simplicity, we omit the parts of the specification that define the register and memory configuration of the ISA, as well as the mapping from field names to bit ranges in the instruction encoding.

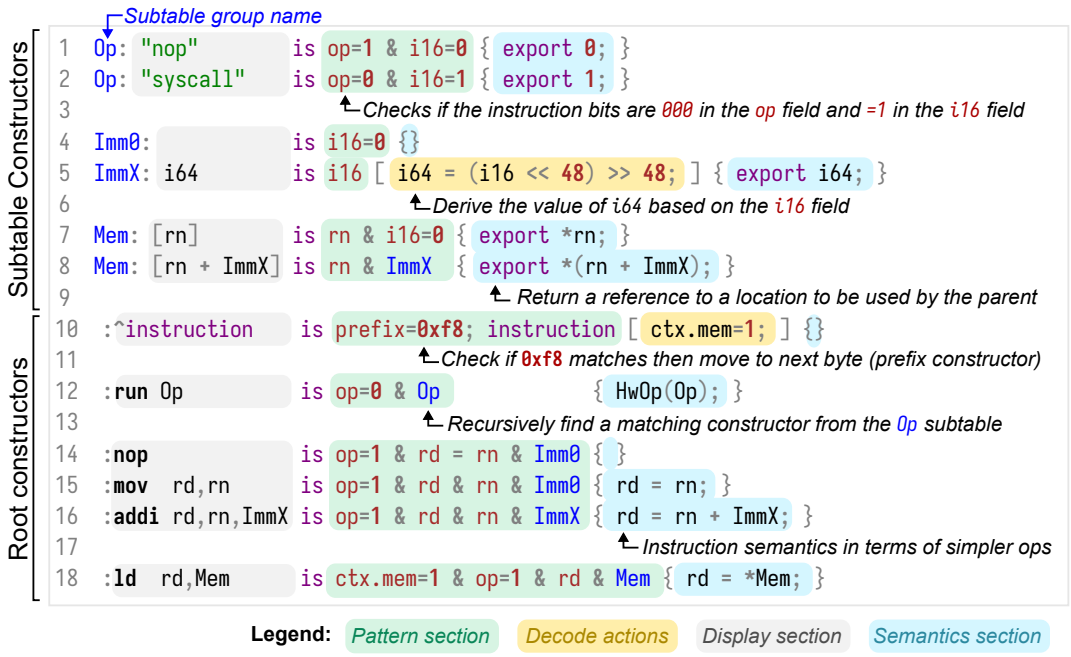


Figure 1: An example of a SLEIGH specification for a simple ISA with a variable length instruction set, such as x86-64.

- **Decode actions**: optional expressions, after the pattern section between '[ ]', that specify how to update the decoder context when the constructor is evaluated.
- **Display section**: defines the textual representation of the instruction used for disassembly.
- **Semantic section**: defines the meaning of the instruction in terms of the simpler ISA-independent operations (Pcode).

Subtable constructors that share a group name (e.g., `Op`, `Mem`) form a table and are responsible for decoding a particular aspect of the instruction, such as operand decoding, and are used at different points during the decoding process. Constructors that do not specify a table name (i.e. those that start with ':'), are implicitly associated with the *root* table, and are the starting point for the decoding process. All other tables are used for decoding only when they appear in the pattern section of one of the root-level constructors (such as `Op` on Line 12).

Employing a SLEIGH specification for decoding begins with two inputs: a sequence of bytes (the instruction encoding) and a set of context bits. Context bits are a SLEIGH-specific mechanism used for two purposes: i) to encode information relevant to decoding that is not directly present in the instruction encoding, such as the current processor mode (e.g., ARM vs. Thumb); and ii) to transfer information between constructors. For example, context bits are often used to alter operand decoding in the presence of prefix bytes.

Given this initial state, the decoder attempts to find a constructor whose *pattern section* matches the current state. Constructors are considered from most to least specific, where specificity is determined by the number of constrained bits in the pattern section[3]. If no constructor matches, the encoding is treated as undecodable or invalid. When a match is found, the decoder applies the corresponding decode actions to update its state. It then recursively evaluates any subtables referenced in the *pattern section* using the updated state. Constructors in each subtable are matched and evaluated in the same way until the instruction is fully decoded. Once decoding is complete, the instruction semantics can be determined by evaluating the *semantic section* of the constructors matched during decoding, potentially substituting subtable references with the values exported by the corresponding constructors.

Example. Consider the `addi r2,r3,0x1234` instruction from before. After extracting the fields from the instruction encoding, the decoder sees: `rd=2`, `rn=3`, `op=1`, `i16=0x1234`, and attempts to find a constructor in the root table matching these values. The `nop`, `mov` and `addi` constructors all match the current opcode (`op=1`). However, `nop` is rejected because `rd`$\neq$`rn` and `mov` is rejected because `i16`$\neq$`0`[4] causing the `addi` constructor to match.

The decoder then evaluates any decode actions (none for the `addi` constructor) and recurses into any referenced subtables. The `addi` constructor has a single referenced subtable (`ImmX`) which has one constructor (Line 5). This constructor has no additional constraints (the `i16` field is unconstrained),

[3]Tie-breaking depends on the declaration order, and non-overlapping constructors can be matched out-of-order.

[4]This requires evaluating the `Imm0` constructor and *backtracking* which is described in more depth in Section 3.1.1.

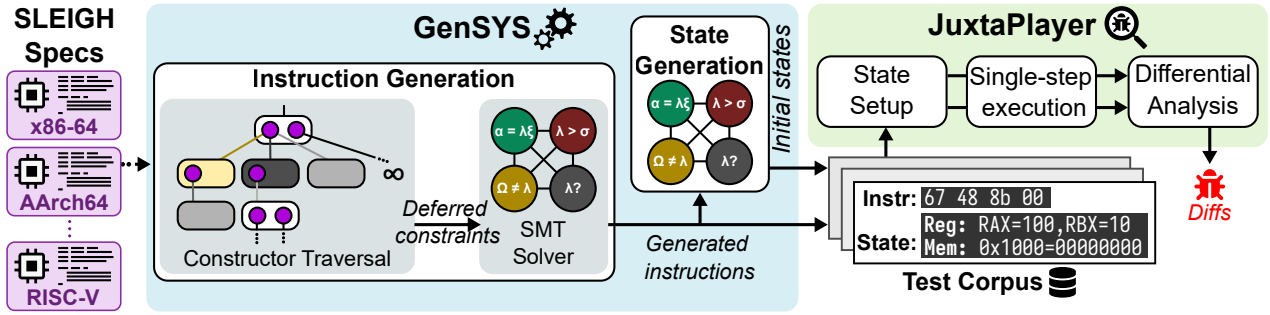


Figure 2: A high-level overview of InSPECtor; the specification-driven test case generation (GenSYS detailed in Section 3.1 & 3.2) and differential testing (JuxtaPlayer detailed in Section 3.3) process to identify bugs or defects in the specification encodings.

so it matches immediately, and the decode action computes `i64=0x1234`. At this point, all operands have been decoded[5], and the target disassembly is obtained by substituting these decoded values into the display section of the matched constructor, yielding `addi r2,r3,0x1234`. Similarly, the instruction semantics are obtained by substituting the decoded operands into the semantic section, resulting in the instruction semantics being: `r2=r3+0x1234`. For a complete SLEIGH DSL description, readers are directed to the resource in [36].

For instruction generation, we are interested in traversing all possible decode paths which requires considering the pattern and decode sections of the constructors we discussed in this section.

3 Methodology

Figure 2 presents an overview of our approach, GenSYS systematically generates decodable instructions as test cases, effectively aiming to cover all instructions recognized by a SLEIGH specification; JuxtaPlayer applies a differential testing strategy, comparing the behavior of executing an instruction in a SLEIGH-based emulator with a hardware reference. The key idea is to employ the SLEIGH-based emulator as a proxy for assessing the correctness of a target specification.

Ideally we would systematically generate and test every unique instruction recognized by a SLEIGH specification. However, testing every possible combination of operand and immediate values is both computationally wasteful and unnecessary, since most operand variations do not affect an instruction's core behavior. Instead, we generate all instructions with unique semantics by traversing a specification's constructors as we detail in Section 3.1, and combine them with a representative subset of operand values: simple defaults, random selections, and values that force operand aliasing within the instruction semantics. In the following, we elaborate on the instruction generation and state generation methods in GenSYS followed by the testing method encapsulated in JuxtaPlayer.

[5]Mapping register fields (`rn` and `rd`) to specific registers is handled by attachments, which is described in Section 3.1.1.

3.1 Instruction Generation

For SLEIGH specifications, generating all instructions with unique semantics is equivalent to generating one instruction for each possible decode path in the specification. We can do this by determining which constraints on the instruction encoding must hold for a given constructor to be selected during decoding, and then solving those constraints to produce a concrete instruction encoding.

Conceptually, this process can be viewed as a constraint satisfaction problem (CSP). In practice, however, efficiently collecting and solving all relevant constraints requires a complex traversal of the decode tree. In particular, the traversal must handle several kinds of constraints, account for implicit constraints introduced by overlapping constructors, manage decode actions that modify decoder state during traversal, and avoid unbounded recursion.

The core of the traversal process is outlined in Algorithm 1. Starting from the root-level table, GETINSTRUCTIONS invokes the CONSTRUCTORTRAVERSAL algorithm, which selects a constructor c for the current subtable t (forking the current state S if there were multiple valid choices), translates all constraints required to match that constructor to symbolic expressions (TRANSLATE), updating S, before recursively repeating the process for any subtables. The state returned by CONSTRUCTORTRAVERSAL contains the complete set of constraints associated with the decode path, which are solved using an SMT solver to produce concrete instruction encoding X. GETINSTRUCTIONS then restores the state to a previous fork and selects a different constructor, repeating this process until every possible path has been explored.

The algorithm's state S consists of three components: i) an instruction bit vector $\mathbf{I} = b_1 b_2 \cdots b_n$; ii) a context bit vector $\mathbf{C} = b_1 b_2 \cdots b_m$; and iii) a collection of deferred constraints. For the bit vectors, each bit is either a fixed value (0 or 1) or symbolic (?). The size of the context bit vector (m) is a constant defined by the specification. For variable-length ISAs, the size of the instruction bit vector (n) is not known

until traversal is complete, instead we define an upper bound[6] and ignore any unreferenced bits after all constructors have been selected.

Algorithm 1 Instruction Generation

```
1:  function GetInstructions
2:      O ← {} ▷ The list of generated instructions (Output).
3:      F ← {} ▷ The set of fork points for unselected paths.
4:      S ← initial state, t ← root subtable
5:      while S ≠ ∅ do
6:          S, f ← ConstructorTraversal(S, t)
7:          F ← F ∪ f ▷ Add new fork points found during
            subtable traversal.
8:          X ← Solve(S) ▷ Get instruction bytes satisfying
            all constraints.
9:          if X is SAT then
10:             O.push(X)
11:         (S,t) ← F.pop()
12:     return O
13:
14: t: subtable to add constraints from
15: S: the current decoder state
16: function ConstructorTraversal(S, t)
17:     F ← {}
18:     if S at traversal limit then
19:         return (UNSAT, F)
20:     c ← CurrentConstructor(S, t)
21:     if S has unselected constructors then
22:         F.push(NextConstructor(S, t))
23:     S.simple ← S.simple ∧ Translate(c.simple)
24:     if S.simple is UNSAT then
25:         return (UNSAT, F) ▷ Terminate traversal early
            if simple constraints are UNSAT.
26:     S.deferred ← S.deferred ∧ Translate(c.cplx) ∧
        Translate(c.attach)
27:     for o_i in FindOverlaps(S, c) do
28:         S.deferred ← S.deferred ∧ ¬Translate(o_i)
29:     S ← ApplyDecodeActions(S, c)
30:     for t_i in Subtables(c) do ▷ Recursively traverse
        subtables and merge constraints.
31:         S, f ← ConstructorTraversal(S, t_i)
32:         F ← F ∪ f
33:         if S is UNSAT then
34:             return (UNSAT, F)
35:     return (S, F)
```

Every bit in the instruction bit vector is initialized as symbolic ($\mathbf{I}_i = ?$ for $i \in 1, 2, \cdots n$), while the context bit vector is initialized to a fixed value determined by the current processor configuration (for our simple ISA specification in Figure 1, it is initialized to 0).

Deferred constraints are additional constraints on the instruction bit vector that cannot be resolved to a fixed value (e.g., inequalities). These constraints are solved simultaneously after the traversal process has completed and all constraints are known. By maintaining known instruction bits separately from deferred constraints, the algorithm can efficiently test satisfiability during traversal and prune large portions of the search space. For example, if a constructor in a subtable requires a particular bit to be 1, but that bit is already known to be 0 from an earlier constraint, then the corresponding subtree can be discarded immediately, significantly improving efficiency. Next, we detail the complex component of the method employed in the instruction generation algorithm design.

[6] Considering the largest possible instruction across all tested ISAs is 120 bits (15-byte x86-64 instructions), $\max(n) = 120$.

3.1.1 Translating Constructor Constraints

The following section describes the method for translating the pattern section of each constructor into a combination of fixed bit patterns and deferred constraints encapsulating the Translate function in Algorithm 1.

Simple constraints. Most constraints involve equality checks between a constant and a range of instruction or context bits. These constraints can be translated directly into fixed values for the corresponding bits in the instruction or context bit vector. If a new constraint is unsatisfiable given the previously fixed values, traversal stops and the algorithm continues with a different set of constructor choices. Otherwise, the newly constrained bits are updated and traversal continues.

Example. Using the specification Figure 1, consider the `:run` constructor (Line 12) and the `Op` subtable (Lines 1–2):

```
1  Op: "nop"     is op=1 & i16=0 { export 0; }
2  Op: "syscall" is op=0 & i16=1 { export 1; }
⋮
12 :run Op       is op=0 & Op    { HwOp(Op); }
```

When the `:run` constructor is selected, the algorithm updates the instruction bit vector so that the `op` field bits are fixed to 0. It then attempts to select a constructor from the `Op` subtable by checking each constructor's constraints against the current state. The `Op:"nop"` constructor requires the `op` field is equal to 1, which conflicts with the current value of 0, so it is immediately discarded. The `Op:"syscall"` constructor is compatible, so it is selected and traversal continues after adding a `i16=1` constraint.

All simple constraints can be reduced to a bitwise mask-and-compare operation. For example, when generating an instruction corresponding to `run syscall`, the final symbolic expression passed to the SMT solver is ($\mathbf{I}$ & `0xfffffe00` = `0x00010000`), where $\mathbf{I}$ denotes the instruction bit vector.

Complex constraints. Beyond simple equality constraints, SLEIGH also allows constructors to use more complex constraints to match instructions. This includes both inequalities and constraints involving non-constant expressions. Unlike simple constraints, there is often no single value that can be assigned to the constrained bits to satisfy the constraint outright. One possible strategy would be to choose *any* value satisfying the constraint and continue traversal; however, such a choice may conflict with constraints introduced by later subtables, preventing valid instructions from being generated. Another option would be to exhaustively explore all possible values, but this would require traversing an excessive number of paths and would generate many redundant instructions. Instead, we elect to check that such constraints are *potentially*

satisfiable (i.e., they do not already conflict with the existing simple constraints) and then add them to a list of deferred constraints, which are solved once traversal is complete.

Once constructors for all subtables have been selected, the full set of constraints for the current instruction is known. To obtain a final instruction encoding, a symbolic expression is constructed from the instruction bit vector together with the deferred constraints. This expression is then passed to an SMT solver (Z3), which attempts to find a concrete assignment to all bits satisfying the combined constraints. If the solver succeeds, it returns a concrete instruction encoding; otherwise, it reports that the constraints are unsatisfiable.

Example. As an example, consider the `:nop` constructor from Line 14 of our example specification in Figure 1:

```
14  :nop   is op=1 & rd = rn & Imm0 { }
```

For an instruction to match this constructor, the `rd` and `rn` fields must be equal. This constraint cannot be translated directly into fixed values for the instruction bit vector, since there are many possible values for `rd` and `rn` that satisfy it. Rather than choosing an assignment prematurely, we add the constraint `rd=rn` to the list of deferred constraints. At the end of traversal, the final expression passed to the SMT solver is:

$$(\mathbf{I} \,\&\, \texttt{0xfffffe00} = \texttt{0x00000200}) \wedge (\mathbf{I}.rd = \mathbf{I}.rn)$$

To optimize the process further, we check whether deferred constraints remain potentially satisfiable given the currently known bits. Suppose there is a deferred constraint `i16 > 0`. The algorithm then checks whether at least one bit in the range corresponding to `i16` is either unknown or equal to 1. If a different constructor requires that `i16=0`, we can immediately reject it due to the `i16 > 0` constraint, which avoids wasting time traversing impossible constructor combinations.

Constraints imposed in attachments. Fields referenced by a constructor may be linked to attachments that control the interpretation of the underlying bits. While most attachments allow all field values, some SLEIGH specifications use attachments to mark certain operand values as invalid. To handle this during instruction generation, invalid values are excluded by introducing appropriate deferred constraints.

Example. A possible set of attachments for our example specification in Figure 1 is as follows:

```
attach variables [rd rn] [_ r1 r2 r3 r4 r5 r6 r7]
```

Whenever `rd` or `rn` appears unconstrained in the specification (e.g., in the `:mov` and `:addi` constructors), its value is interpreted according to the attachment mapping. For example, `rd=1` maps to `r1`; `rd=2` maps to `r2`, and so forth. In this example, an attachment index of 0 has no associated register, indicated by the '_' operator. To account for this, we treat such cases as implicit complex constraints: `rd`$\neq$`0` and `rn`$\neq$`0`. For instruction generation, these constraints are handled in the same way as other complex constraints discussed previously.

Implied constraints from overlapping constructors. The behavior of an instruction is primarily determined by its 'opcode' field,[7] which specifies the operation performed on the instruction's operands. In general, two instructions with the same opcode have the same semantics regardless of their operand values. However, some instructions can be represented as a specific sub-encoding of a more general instruction. For example, copying a value between registers (e.g., `mov r0,r1`) is semantically equivalent to adding zero to a source and storing the result in a destination register (e.g., `addi r0,r1,0`). These instructions are often referred to as *pseudo-instructions*, and are present in many ISAs.

Although SLEIGH specifications could ignore pseudo-instructions and always decode them as their generic counterparts, modeling them explicitly benefits both human and machine analysis. A more specific textual representation improves disassembly for human analysts, and defining specialized semantics can simplify both static and dynamic analysis.

In addition to pseudo-instructions, some ISAs include instructions with more complex operand encodings that require special handling during decoding. For example, in both x86-64 and MSP430, instruction encodings include 'addressing mode' bits that alter the interpretation of operand fields.

To model such encodings, SLEIGH provides a mechanism known as constructor specialization, which allows multiple constructors within the same subtable to match the same instruction encoding. Recall (Section 2), the SLEIGH decoder prioritizes matching constructors from most to least constrained. Consequently, for an instruction to match a less constrained constructor, it must also avoid matching any overlapping constructors that would be considered first. To model this behavior, we treat the target constructor as if it carried an additional implied constraint requiring all preceding overlapping constructors to not match.

Example. Consider Lines 14–16 of Figure 1:

```
14 :nop             is op=1 & rd = rn & Imm0 { }
15 :mov  rd,rn      is op=1 & rd & rn & Imm0 { rd = rn; }
16 :addi rd,rn,ImmX is op=1 & rd & rn & ImmX { rd = rn + ImmX; }
```

To generate a `mov` instruction, we must avoid matching the `:nop` constructor. Since the only difference between the two constructors is the constraint `rd=rn`, we add its negation, `rd`$\neq$`rn`, to the list of deferred constraints for the `:mov` constructor.

In rare cases, there are no unique constraints that can be added to distinguish between two overlapping constructors. For example, consider the `:addi` constructor in the example specification. This constructor overlaps with both `:nop` and `:mov`. Similarly, we can avoid matching `:nop` by adding

[7] To keep instruction encodings compact, the full opcode may be split over multiple fields depending on the ISA.

the constraint `rd≠rn`. However, at the constructor level, the constraints for `:mov` and `:addi` are identical.

The only difference between the two constructors is the final subtable (i.e., `Imm0` and `ImmX`). Since subtables are evaluated only after a constructor has been selected, it may seem impossible for `:addi` to match. However, SLEIGH supports *backtracking*; if no constructor matches in a later subtable, the decoder may return to an earlier point in the process and try a different constructor.

Therefore, to ensure that `:addi` is selected, we must force the SLEIGH decoder to backtrack from the `:mov` constructor. To achieve this, the generation algorithm inspects all constructors in the `Imm0` subtable and adds additional constraints that ensure that no constructor matches. In this example, the `Imm0` constructor requires `i16=0`, so we add the constraint `i16≠0`. If the subtable contained multiple constructors, we would instead need a combined constraint ensuring that all of them fail to match. The final set of constraints the algorithm solves for the `:addi` constructor is:

$$(\mathbf{I} \,\&\, \texttt{0x0000fe00} = \texttt{0x00000200}) \wedge \quad \textit{Match :addi}$$
$$(\mathbf{I}.rd \neq \mathbf{I}.rn \vee \mathbf{I}.i16 \neq 0) \wedge \quad \textit{Avoid :nop}$$
$$(\mathbf{I}.i16 \neq 0) \quad \textit{Avoid :mov}$$

Notice that when we add constraints to resolve the overlap with the `:nop` constructor, the overlap can be resolved by negating the simple constraints *or* by backtracking, maximizing the flexibility the solver has to generate an encoding, allowing `addi r0,r0,1` to be a valid solution.

Self-referencing constructors and decode actions. SLEIGH allows constructors to be *self-referencing*, meaning that a constructor may reference the same subtable to which it belongs. This functionality is used for two main purposes: i) *phased decoding* and ii) optional *prefix parsing*. In both cases, the self-referencing constructor must be able to influence subsequent steps of the decoding process. To support this, constructors may specify *decode actions* in square brackets after the pattern section. Each decode action evaluates an arithmetic expression, possibly over context and/or instruction bits, and writes the result to a context field.

Phased decoding. For phased decoding, context fields are used to split the decoding process into multiple phases, where previous phases manipulate other context bits which are used as constraints in later phases.

During instruction generation, this mechanism is straightforward to handle; at any point in the traversal, the phase has a fixed value, so only constructors for the current phase need to be considered. Phased decoding is not used in our example specification, instead we provide the following example (inspired by the AArch64 specification) to explain.

Example. Consider the case where the immediate field is split across the instruction encoding:

```
:^instruction   is phase=0 & immhi & immlo & instruction
                   [ phase=1; imm=(immhi<<8 | immlo); ] {}
:mov rd,rn      is phase=1 & op=0xf & rd & rn & imm=0
                   { rd = rn; }
:addi rd,rn,imm is phase=1 & op=0xf & rd & rn & imm
                   { rd = rn + imm; }
```

Assuming the `phase` context field is initialized to 0, only the first constructor matches the current context. The instruction generation algorithm therefore selects it and updates the context according to the decode actions. When the algorithm subsequently resolves the `instruction`[8] subtable, it uses the updated context, where `phase=1`. Since the phase value is now 1, only the second and third constructors match, allowing them to act as if they were separate subtables. The `:mov` constructor demonstrates the usefulness of phased decoding, where it is able to define a constraint on the entire immediate value instead of separate constraints on each subfield.

Prefix parsing. Constructors used for prefix parsing are more involved to translate. Since prefix constructors also consume part of the instruction bit vector, the algorithm must track the offset within the instruction encoding, and adjust any subsequent constraints. Additionally, since prefix bytes can be optional and repeated (e.g., x86-64), unlike phased parsing, prefix constructors cannot be easily separated from regular opcode decoding, which can lead to implied constraints from overlap resolution. Further, to avoid generating arbitrarily long encodings from repeated prefixes, we must incorporate a traversal limit during the traversal process.[9]

Example. Consider the `:ld` constructor:

```
10  :^instruction is prefix=0xf8; instruction [ctx.mem=1] {}
18  :ld  rd,Mem   is ctx.mem=1 & op=1 & rd & Mem { rd = *Mem }
```

For the `:ld` constructor to match, instead of one of the arithmetic instructions with the same opcode, the context field `ctx.mem` must be set to 1. This occurs only if the *prefix* constructor (Line 10) is matched first; that constructor adds the constraint that the instruction must begin with the prefix byte (`0xf8`) and updates the context field `ctx.mem` to 1. Consequently, when the instruction generation algorithm tries to generate an encoding for the `:ld` constructor, it emits the following constraint:[10] $\mathbf{I} \,\&\, \texttt{0x0000fe00ff} \neq \texttt{0x00000400f8}$.

Dynamic context. If the result of a decode action depends on unconstrained parts of the instruction encoding, the corresponding parts of the context bit vector become partially symbolic. Because the only source of symbolic values is the instruction bit vector, each value can be expressed as a function of instruction bits and constants. Any later constraint on

[8] Recall that the `instruction` identifier is a special identifier that matches the root subtable—constructors prefixed with just ':'.

[9] Given that physical hardware will always have an upper limit on instruction length, this should never prevent us from generating a valid instruction.

[10] Constraints simplified; in practice, additional constraints are required for overlap resolution and attachments.

those context bits can therefore be translated into a constraint on the instruction bit vector. These translated constraints are added to the deferred-constraint set and solved after traversal.

Example. Consider the following extension to our example specification:

```
Rd2: ctx.rd2     is ctx.rd2 < 7 & ctx.rd2   { ... }
Mem2:            is rn & ImmX               { ... }
:ld2 rd,Rd2,Mem  is op=2 & rd & Rd2 & Mem & Mem2
                    [ctx.rd2 = rd+1; ]      { ... }
```

The instruction generation algorithm must track that `ctx.rd1` is equivalent to `rd+1`, and ensure that a subsequent constraint on `ctx.rd1` is mapped to a corresponding constraint on `rd`. For example, the `ctx.rd1 < 7` constraint in the `Rd2` subtable becomes `rd+1 < 7`.

3.1.2 Avoiding Excess Constructor Combinations

SLEIGH specifications use attachments to map the numerical value of a field to a specific register. However, more complex relationships between fields which cannot be easily expressed with attachments are sometimes required. In such cases, specifications resort to mapping field values through separate constructors, which exponentially increases the number of constructor combinations. For example, ARM allows any combination of the registers `r0` to `r15` to appear in the register lists used by the load/store-multiple instructions `LDMIA` and `STMIA`. The corresponding decoding specification (Figure 3) requires traversing approximately 2^{16} paths to cover all operand combinations. To avoid generating an instruction for every combination, the algorithm tracks which constructors have already been covered and prunes them during subtable traversal when this type of explosion is detected (*traversal limit* on Line 18).

```
1 rlist_15: rlist_14,r15  is b15=1 & rlist_14 {...}
2 rlist_15: rlist_14      is b15=0 & rlist_14 {}
3 rlist_14: rlist_13,r14  is b14=1 & rlist_13 {...}
4 rlist_14: rlist_13      is b14=0 & rlist_13 {}
5 # ... Repeated for rlist_13 to rlist_01
6 rlist_00:               is b00=0            {}
7 rlist_00: r0            is b00=1            {...}
8 rlist: {rlist_00}       is rlist_00         {...}
```

Figure 3: Register list operand decoding in the ARM SLEIGH specification (e.g., for `LDMDA r0, {r1,r2 ...}`): Each register in the list can be either present or absent and there are 16 registers (`r0` to `r15`) leading to 2^{16} combinations.

3.1.3 Solving Instruction Encodings

After returning from ConstructorTraversal, all constructors have been selected. However, when we attempt to solve the constraints to find a concrete instruction encoding (Line 8), there can still be several bits in the encoding that remain unconstrained, or only partially constrained (e.g., `imm > 0`). These 'free' bits correspond either to generic operands or to unused bits that do not affect the instruction. As a result, each instruction typically has a range of valid encodings. Exhaustively generating and testing every combination of these free bits would reintroduce the same exponential explosion we are trying to avoid. Instead, the algorithm generates a smaller representative set of solutions that covers the most *interesting* cases.

To do this, Solve attempts to identify possible edge cases in the instruction encoding. It uses Z3's support for minimizing and maximizing values [5] to generate test cases at the minimum and maximum encodings permitted by the constraints, which helps expose potential off-by-one errors in partial constraints. For example, for the `:nop` constructor (Line 14 of our example specification), given the complex constraint `rd=rn`, Z3 generates encodings corresponding to `rd=rn=0` and `rd=rn=7`.

Next, analysis of the instruction's semantics identifies any registers that could alias with operands derived from the unconstrained bits. Additional constraints are then added to the instruction bit vector to force operands to match the aliasing register, generating targeted cases for each potential alias.

3.2 State Generation

The behavior of an executed instruction depends on the state of the processor. Therefore, to comprehensively test a specification, we need to explore not only changes to the instruction bytes but also to the initial input state. The state is made up of several components, including the values of ISA-level registers, memory, and processor-specific internal state (e.g., model-specific registers, caches, and micro-architectural resources). For specification testing, we assume the internal state is fixed and is reset between instruction executions. However, both ISA-level registers and memory are allowed to vary.

Algorithm 2 State Generation (from Pcode edge cases)

```
1:  function GenInputStates(P) ▷ P: List of Pcode operations for an instruction.
2:      G ← {} ▷ Output G: Set of edge case triggering input states.
3:      S ← initial symbolic state
4:      for p_i in P do
5:          (a,b) ← S.get(p_i.inputs) ▷ Get the current symbolic value of the inputs.
6:          o ← Symbolise(a, p_i.op, b) ▷ Symbolically execute p_i.
7:          for e in EdgeCases(p_i.op) do
8:              s ← Solve(Assert((a,b,o) triggers e)) ▷ Find a concrete input s triggering e.
9:              if s is SAT then
10:                 G ← G ∪ s
11:         S.set(p_i.output, o) ▷ Update the symbolic state with the output of p_i.
12:     return G
```

Group	Example Pcode Operations	Edge cases
Addition/Subtraction	INTADD, INTSUB	Overflow/underflow, result negative, result zero.
Multiplication/Division	INTMULT, INTDIV, INTSDIV	Operands negative/zero.
Zero/Sign Extension	INTZEXT, INTSEXT	Input all ones or all zeros, output non-negative and negative.
Boolean	BOOLXOR, BOOLOR, ...	Result true/false.
Bitwise	INTXOR, INTOR, ...	Result zero/non-zero.
Shifts	INTLEFT, INTRIGHT, ...	Oversized shift operand, overflowing result, sign-bit set.
Comparisons	INTEQUAL, INTLESS, ...	Operands equal or ± 1.
Overflow conditions	INTCARRY, INTBORROW, ...	Result true/false.
Floating point	FLOATADD, FLOADSUB, ...	Operands with interesting floating point values (e.g. NaN).
Branches	CBRANCH	Condition is true and path constraints for future operations.

Table 1: Edge cases used for generating initial interesting input states from Pcode operations.

Although many specification bugs are discoverable with an arbitrary initial state, certain bugs may manifest only when an instruction is executed under extremely specific initial conditions. To allow us to test these cases, Algorithm 2 is used to generate interesting input states by identifying potential edge cases from the semantics section of the specification. We exploit the fact that SLEIGH breaks instructions into simpler operations (Pcode) to express each edge case as an assertion in terms of the inputs and/or output of a specific operation. By systematically defining edge cases for each Pcode operation and generating initial states that trigger each edge case, it is possible to find discrepancies even when the semantics are only subtly incorrect.

Example. Consider an `ADD` instruction that also needs to update various arithmetic flags, for example on x86-64 the Pcode for `ADD r0,r0,r1` is as follows:

```
OF = r0 IntCarry r1
r2 = r0 IntAdd r1
ZF = r2 IntEqual 0
NF = r2 IntSignedLessThan 0
```

The `ADD` instruction is broken into four Pcode operations. Consider the second operation, `r2 = r0 IntAdd r1`, where `r0` and `r1` are *inputs* to the instruction and are replaced with symbolic variables a_{r0} and b_{r1} (Line 5 of Algorithm 2), the EDGECASES method returns the following three assertions[11]:

$$a_{r0}+b_{r1} \text{ overflows}, \qquad a_{r0}+b_{r1}=0, \qquad a_{r0}+b_{r1} \text{ s}< 0$$

Values for a_{r0} and b_{r1} are obtained by symbolically solving the assertions using an SMT solver (e.g., for the third assertion $a_{r0}=0, b_{r1}=-1$). After saving valid edge-case triggering inputs, the algorithm updates the symbolic state (Line 11) such that, $\texttt{r0}=a_{r0}+b_{r1}$. This means, when generating edge cases for the later Pcode operation `ZF = r2 IntEqual 0`, we actually generate edge cases for $a_{r0}+b_{r1}=0$.

All assertions are solved independently; the state generation process does not attempt to find initial input states that simultaneously solve multiple assertions. This avoids an exponential explosion of assertion combinations, many of which would be unsatisfiable. In practice, complex input states are still generated from later Pcode operations because any preceding Pcode operations that modify inputs to the operation are incorporated in symbolic expressions. In some cases, previous operations can restrict the possible values of the operands; this makes the edge cases unsatisfiable and causes no corresponding input state to be generated.

For many ISAs, instructions may update status register bits (flags) when specific conditions occur while executing the instruction. In SLEIGH specifications, these conditions are checked using individual Pcode operations, which will have edge-case-triggering inputs generated for them. This can result in the state generation process producing duplicate input states, since the conditions checked by the instruction are often the same as the edge cases defined for state generation.

Example. In the example code above, the same initial input state that satisfies $(r0+r1 \text{ s}< 0)$ (edge case for addition), also satisfies $(r2=-1)$ (edge case for comparisons) since $r2=r0+r1$. However, this redundancy is useful because it ensures that missing or incorrectly implemented semantics are still tested. In cases where exact duplicates are generated, they are automatically removed before the test cases are executed.

A summary of the edge cases used for state generation is provided in Table 1. The motivation for these edge cases and how they interact with more complex Pcode operations such as branches and memory accesses is detailed in Appendix C.

3.3 Differential Testing Environment

We use a differential testing strategy to discover bugs in SLEIGH specifications using the generated test cases. The strategy is implemented in JuxtaPlayer, which executes the test cases on two different systems: i) an emulator derived from the SLEIGH specification under test; and ii) hardware-based reference systems.

- **SLEIGH-Based Emulator Configuration.** JuxtaPlayer uses Icicle [8], a SLEIGH-based emulator, to execute Pcode operations from the SLEIGH specification.

[11] s< represents a *signed* less-than operation.

	Generation Time (s)			Generated Tests		*Constructor*	Missing Constructors			
Architecture	**Traversal**	**Solving**	**State Gen**	**Instructions**	**States**	***Coverage***	**N/A**	**Redun.**	**Incom.**	**Bugs**
x86-64	491.99	1019.56	425.45	71,852	712,668	97%	77	5	0	4
AArch64	0.24	90.09	573.02	25,031	123,031	97%	104	41	4	0
ARM/Thumb	1387.26	691.99	59.11	30,709	207,999	96%	50	79	2	3
RISC-V	0.07	39.69	6.76	9,725	38,481	99%	4	6	1	0
MSP430	0.09	46.10	557.15	7,436	124,712	90%	1	79	0	0

Table 2: Summary of the generation process execution for each architecture. For missing constructors, *N/A*: Not applicable (only used different SLEIGH config), *Redun.*: Redundant constructors, *Incom.*: Unreferenced constructors due to incomplete specifications, Bugs: root-caused bugs in the specification.

- **Hardware-Based Reference (KVM).** Our goal is to ensure SLEIGH specifications accurately reflect the behavior of real hardware, hence, we use hardware as an oracle. We support hardware-assisted virtual machines (VMs) for x86-64, AArch64 and ARM/Thumb
- **Hardware-Based Reference (Debugger).** Available MSP430 and RISC-V hardware did not support virtualization extensions, therefore using KVM-based VMs was not possible. For the MSP430 reference, we use an MSP430FR5969 microcontroller and use a JTAG hardware debugger. For RISC-V we used the BeagleV®-Ahead with a quad-core Xuantie C910 processor implementing RV64GC, including the 0.7.1 draft version of the vector extension, and ptrace on Linux for debugging.

Each of system supports single-step instruction execution, direct reading and modification of the system state needed for differential testing, and exception tracking during execution. Further details on the configuration of each system and the supporting infrastructure are available in Appendix C.

Before an instruction is executed, the environment is configured to match the initial state defined by the test case. This includes a region of memory containing the instruction bytes with the execute permission set. Registers and other memory regions are then initialized according to the values produced by the state generation algorithm. The instruction pointer (i.e., RIP) is positioned at the instruction address, and the system is set up to execute a single step. The execution may succeed or raise an exception. In either event, the final system state is recorded, including modified memory values, registers, and any exception information, for later comparison. Notably, for performance, instead of mapping the entire address space, JuxtaPlayer lazily maps the memory whenever a memory exceptions occur and replays the instruction.

4 Evaluation

We focus our evaluation on five important SLEIGH specifications (x86-64, AArch64, ARM/Thumb, RISC-V, and MSP430) to assess coverage, discrepancy patterns, and bug prevalence across diverse ISA designs. This section summarizes the results for all ISAs.

All instructions were generated using the SLEIGH specification from Ghidra 10.3.[12] GenSYS is executed on a Threadripper Pro 5995WX with 64 GB RAM[13]. Through the evaluation, we seek to answer the following questions:

- How generalizable is InSPECtor in traversing the ISA variants of SLEIGH specifications, given the diverse specification styles used by different authors and contributors? (Section 4.1)
- How effective is InSPECtor at finding bugs in SLEIGH specifications? (Section 4.2)
- What is the nature and impact of the bugs found by InSPECtor? (Section 4.2 and Appendix C)
- How effective are the techniques employed by InSPECtor? (Section 4.3)

4.1 Test Case Generation

A means to objectively assess the test coverage of a given specification is necessary for assurance and evaluation of the testing method to understand its effectiveness. Analogous to considering code-coverage metrics as a proxy for a bug-based metric in software testing [16, 21] or fuzzing [12], we consider measuring **constructor coverage** as a proxy for the testing coverage of a given specification. Given that the fundamental unit of syntax in SLEIGH is a constructor, constructor coverage is a meaningful evaluation metric. We compute constructor coverage as:

$$\text{Constructor Coverage} = \frac{\text{Constructors Used in Generation}}{\text{Total Constructors in a Spec.}}$$

Further, to demonstrate the efficiency of the generation process, we also report the test **Generation Time**.

Generation Results. GenSYS was used to generate instructions and states for x86-64, AArch64, ARM/Thumb, RISC-V and MSP430. The results are summarized in Table 2. These results represent unique constructor combinations, i.e., instructions, including variants, and input states covering the edge cases defined in Section 3.2.

[12] Commit: 104d9592485c2fb6c7ab5177c71d8d70140a3d03

[13] The traversal process is single threaded, while state-generation was run in parallel using 8 threads.

Architecture	Execs/s	Raw	Groups	Unimplemented	Alignment	Exception	Flags	Float	PC	State
x86-64	176,404	349,897	18,827	9159	653	2467	3627	605	257	2059
AArch64	3,723	52,833	7,342	5127	21	1451	5	426	0	312
ARM/Thumb	3,572	78,251	9,785	4652	12	4182	32	376	10	521
RISC-V	1,012	15,682	937	322	0	373	8	222	1	11
MSP430	0.164	93,043	2,028	145	0	468	163	0	68	1184

Table 3: Discrepancies found by running JuxtaPlayer on the testcases generated by GenSYS. *Raw:* total number of discrepancies (including variations to input state). *Groups:* unique constructor combinations for which at least one discrepancies was found. Each of these groups was then assigned a category based on observing: the set of differences (exception state and memory/register values) and the operations in the SLEIGH semantics for the constructor combination.

Although the instruction generation process employed by GenSYS exhaustively explores the SLEIGH specification, a total of 460 constructors remained without any generated test cases. We manually investigated each missing constructor and categorized them in Table 2, in summary:

- **Not Applicable.** Many of the missed constructors correspond to processor modes or ISA extensions that are not enabled in our configuration (e.g., x86-64 real/protected-mode semantics and AArch64 pointer-authentication support). Because these features are unavailable at runtime, the associated constructors can never be matched and can be ignored.
- **Redundant.** A large proportion of the missing constructors correspond to cases where they completely overlap with a set of more specific constructors. While these redundant constructors are benign and have no impact on the specification's correctness, they can be removed to simplify the specification and improve maintainability.
- **Incomplete.** Some parts of the specification are incomplete, and the missing constructors correspond to cases where a subtable constructor is fully implemented, but the parent constructor is missing.
- **Specification Issues (Bugs).** A small set of the missing constructors represent issues in the SLEIGH specification.

4.2 Differential Testing Analysis

Across all architectures, JuxtaPlayer discovered 589,713 discrepancies between the output states of the SLEIGH-based emulator and the hardware reference system corresponding to 38,920 unique constructor combinations after grouping. However, many of these stem from hardware/processor-specific instructions (see Appendix C), or from instructions that are not fully implemented in the specification[14]. Analyzing the discrepancies enabled us to identify 125 unique bugs. In the following sections, we describe the discovered discrepancies, the analysis process and the specification bugs. We also present an analysis of root causes and sources of errors we uncovered along with bug examples.

[14] SLEIGH allows custom Pcode operations as placeholders for instructions during decompilation. Since no Pcode is associated with these operations, they cannot be emulated and analyzed using differential testing.

4.2.1 Discrepancy Analysis and Specification Bugs

In general, discrepancies correspond to either bugs or SLEIGH specification limitations. In Table 3, discrepancies are first grouped by unique constructor combinations and then assigned to categories we describe here, using an ordered set of heuristics. This categorization serves two purposes. *First*, it separates discrepancies that are unlikely to correspond to actionable SLEIGH bugs—for example, those involving unmodeled hardware state or undefined behavior—from discrepancies that more directly indicate errors in decoding or instruction semantics. *Second*, it provides a consistent link between the observed form of a discrepancy and the mechanism used to detect it, which makes the results easier to interpret.

- **Unimplemented:** Discrepancies corresponding to either: unimplemented behavior—caused by placeholder semantic operations in the SLEIGH specification; or unsupported behavior—use of architectural state not represented in our model, such as model-specific registers (MSRs) or other hardware state not defined at the ISA level. These are detected by the presence of placeholder operations (e.g., `unimpl` blocks or custom `pcodeop` definitions), or the use of unmodeled registers in the SLEIGH semantics. Unimplemented behaviors often arise for recently added ISA features, or instructions with difficult semantics to directly express in Pcode. Unsupported behavior typically corresponds to hardware-specific operations that may depend on implementation-specific hardware details rather than ISA-defined semantics.
- **Exception:** Discrepancies in the type or metadata (e.g., faulting address) of an exception raised by the SLEIGH semantics relative to the hardware reference. It also captures cases caused by ISA feature mismatches (e.g., instructions from some ISA extensions not present on the hardware reference, but included in the SLEIGH specification). Exception mismatches are also frequently related to unpredictable/undefined behavior. For example, on ARM/Thumb some instructions can be encoded with the

program counter (PC) as a destination register, which is listed as unpredictable behavior in the architecture. The SLEIGH specification treats this as ignoring the operation in many cases, while the hardware reference raises an exception. Unfortunately, the variability of exception sources makes it difficult to automatically separate out true issues in this category from unpredictable/undefined behavior.

- **Alignment:** Discrepancies related to alignment issues. They are separated out as a special case of exception discrepancies since these are generally unfixable without adding additional semantics to SLEIGH—see Section 5.1.
- **Flags:** Discrepancies in architectural flags, such as condition-codes or status bits. These are detected when the only observed state difference is in the flag register set. We isolate this class because flag behavior for some architectures is frequently related to undefined or unpredictable cases, and therefore benefits from a separate analysis.
- **Float:** Discrepancies in registers or memory that arise along an execution path containing floating-point Pcode operations. These are detected by tracking whether the corresponding SLEIGH semantics use floating-point operators. This category is separated from general state discrepancies because some differences may reflect limitations of the Pcode floating-point model rather than errors that can be fixed by modifying the SLEIGH specification.
- **PC:** Discrepancies in the final program counter, including incorrect branch targets or advancement to the next instruction—e.g., from decoding the wrong instruction length on x86-64 and MSP430. These are detected when execution ends at a program counter different to the oracle. Such cases are strong indicators of decoding errors and therefore often correspond directly to SLEIGH specification bugs.
- **State:** Any remaining discrepancy in registers or memory not captured by the preceding categories. This is the residual class used when the final architectural state differs but no more specific heuristic applies. In practice, these cases are often the most interesting, as they may indicate incorrect semantics, missing side effects, or incorrect handling of particular operand forms.

Notably, the categories above serve as a prioritization mechanism for analysis as much as a descriptive taxonomy since discrepancies in classes are difficult to directly associate with a specific error in the SLEIGH specification. While each discrepancy corresponds to a possible bug, each bug usually results in many different discrepancies

To analyze the discrepancies within a group, we manually aggregate discrepancies based on mnemonic, semantics, and overlapping constructors, then selected a representative example to study. Time to triage a unique bug was approximately 4 hours on average; the time varies significantly depending on root-cause complexity. This was followed by a considerable time to craft, test and document suitable fixes for Ghidra maintainers. This process enabled us to identify 125 unique bugs (where a bug corresponds to a unique root-cause fixed in an ISA).

Architecture	Decoding	Semantics Ordering	Incorrect Semantics	Aliasing	Total
x86-64	16	0	12	4	32
AArch64	0	1	22	0	23
ARM/Thumb	0	0	15	2	17
RISC-V	2	1	12	3	18
MSP430	2	6	26	1	35
Total	16	9	87	9	125

Table 4: Summary of bug types by architecture.

Consequently, the bug count is not the number of buggy instructions, which would be an inflated number. Tables for the unique bugs based on our analysis of discrepancies for x86-64, AArch64, ARM/Thumb, RISC-V and MSP430 ISAs are contained in Appendix C. To characterize the types of SLEIGH issues encountered across the specifications in the bugs discovered, we categorize the bugs and summarize the results in Table 4. We elaborate on the classes of bugs in I-1–I-15 in more detail, with bug examples, and and provide additional bug case studies in Appendix C.

4.2.2 Decoding Issues

In general, decoding issues occur when incorrect constructor constraints prevent the correct instruction implementation from being selected. We detail these in the following.

I-1 Redundant and out-of-order prefixes. Instruction prefixes are one of the most complex aspects of x86-64 instruction encoding and causes discrepancies between hardware vendors (see Appendix C), security-critical CPU bugs [27], and severe performance issues [13]. Similarly, the x86-64 SLEIGH specification contains many prefix-related decoding issues. x86-64 instructions allow redundant and duplicate prefixes; however, the SLEIGH implementation assumes that at most one of each prefix is present. For example, when multiple operand-size (66) or address-size (67) override prefixes appear, the specification computes the final address/operand size incorrectly, leading to discrepancies.

Additionally, the 64-bit extension to the ISA introduces a new class of prefixes (the REX prefix) with different decoding rules compared to legacy prefixes. According to the ISA reference, there must be at most one REX prefix present, and it must follow all legacy prefixes. However, both the Intel and AMD CPUs tested are more lenient during decoding; any REX prefixes that precede legacy prefixes are ignored, and they allow multiple REX prefixes, only using the final one. The SLEIGH specification does not replicate this behavior; instead, it rejects all instructions where there are multiple or out-of-order REX prefixes.

While compilers avoid emitting unnecessary prefixes, thus reducing the impact of this issue on real-world code, such

prefixes occasionally appear in handcrafted assembly for low-level performance optimizations. For example, OpenSSL uses redundant prefixes to align code to micro-architecturally friendly boundaries [2].

I-2 Context handling issues. Historically, the x86-64 SLEIGH specification used a single context field for both indicating whether long mode (64-bit) instructions were decodable and for tracking the current address size. This design assumed that long mode could be inferred from the address size, because it has a different default setting compared to other processor modes. However, including an address-size override prefix (`67`) changes the current address size, causing the specification to mistakenly assume that the processor is not in long mode. Although the specification was later refined to introduce a dedicated context bit for long mode, many long mode instructions still reference the operand-size bit, resulting in discrepancies.

I-3 Overlapping constructors. Some constructors are unreachable because every instruction encoding that matches them is instead matched by a different constructor (with different semantics). Further analysis revealed that many overlapping constructors are *under-constrained*. This bug causes the wrong instruction semantics and disassembly to be used for the instructions associated with an unreachable constructor.

I-4 Over constrained constructors. Several constructors have additional constraints that are not present in the underlying ISA. For example on x86-64, two constructors require a specific bit in the decoder context to be matched. However, this bit is only set in 32-bit protected mode, even though the associated instructions are also valid in 64-bit long mode. This issue likely stems from a mismatch in the way context bits are used in the SLEIGH specification. On real hardware, the protected mode bit is set in long mode, but the SLEIGH specification only sets an equivalent bit when configured for 32-bit mode. The impact of this bug is that the corresponding instructions are incorrectly treated as invalid in long mode.

I-5 Constructor ordering. For x86-64 specifically there were also several issues related to constructor ordering. Two unreachable constructors, corresponding to 64-bit variants of instructions, share the same encoding as their 32-bit counterparts, except for a mandatory `REX.W` prefix. However, the 32-bit variant allows the prefix to be present and is always matched first, preventing any encoding from matching the 64-bit variant. This bug likely originates from unclear ordering rules. Since the 64-bit variant is not strictly more constrained than the 32-bit variant, the decoder defaults to the declaration order. This issue causes the wrong register sizes (32-bit instead of 64-bit) to be used for the instruction, resulting in incorrect disassembly and semantics.

4.2.3 Semantics Ordering Issues

I-6 Semantic ordering issues arise when correct outcomes are dependent on a specific order of operations which are not followed. This was common in the MSP430 ISA which makes heavy use of addressing modes to augment its basic instructions. These modes include direct and offset indexing, and address auto incrementing among others. All of which are available to the `MOV` instruction leading to cases such as `MOV.W @R7+, 0x0(R7)`. The MSP430 was found to execute this instruction by resolving and incrementing the source register, before resolving the destination, performing the instruction's operations with the pre-incremented value, and writing back the result. This causes loading the destination `0x0(R7)` to be impacted by the increment of the source `@R7+`. This differed from the SLEIGH implementation which always performed the increment as the last operation of the instruction. This also occurred in pseudo-instructions such as `POP.W SP` (`MOV.W @SP+, SP`) even though they had separate Pcode semantics.

Semantic ordering issues were also found outside of addressing modes in AArch64. The `NGCS` instruction's output was written to the destination register, and then the flags were calculated off that register's value. However when using the zero register as the destination register no write is performed, and the flags are always incorrectly calculated from the fixed value of zero.

4.2.4 Incorrect Semantics Issues

Incorrect or missing semantics bugs appear when an implementation uses the incorrect operations, or does not implement all behaviors of an instruction. This was the largest category of bugs, and also contained the most *subtle* and unique mistakes. For example, in AArch64 the `BCAX` instruction was implemented with an *or* operation instead of an *xor*, and `STLRB` stored a word sized value instead of a byte. In addition to these subtle issues, there were several wider classes of issues.

I-7 Missing floating point semantics. RISC-V floating point extensions encode smaller precision floats as NaN-boxed inside larger registers by setting all the unused upper bits to 1's. But, the SLEIGH specification does not use this encoding for single precision floats when writing to the registers, nor does it propagate un-NaN-boxed values as NaN's, causing all initial floating point test cases to fail. The extra logic that fixing this issue added to the start and end of all floating point instructions was not able to be filtered out by Ghidra's decompiler, causing the decompilation to become bloated.

I-8 Default values and behaviors. RISC-V defines specific result values for arithmetic exceptions. On division by zero, the division instruction should return $2^{width} - 1$, and the remainder should be the dividend. There was no zero check before the division, resulting in this behavior being dependent on the execution environment default.

I-9 Incorrect register lists. To implement instructions operating on groups of registers in SLEIGH, recursive subtables are used where the different constructors conditionally include the Pcode semantics for that register. This occurs in

MSP430, AArch64, and ARM/Thumb, with MSP430 having an issue where it did not correctly handle register wrapping, and ARM/Thumb having an issue where the registers were pushed in the wrong order. However, this style of implementation inherently requires duplicated sets of constructors based on the number of registers, requiring the same fix to be applied to each constructor in the set.

4.2.5 Aliasing Issues

I-10 The SLEIGH specification implements instructions using multiple Pcode operations. Consequently, when source and destination operands refer to the same register or memory location (i.e., they alias), special care is required to preserve the atomicity of input values. This issue was frequently handled incorrectly in SIMD instructions (x86-64, ARM/Thumb), where the part of the destination register is often modified before the source register had been completely read.

For RISC-V this occurs in the Control and Status Register extension in the `FSCSR`/`FSFLAGS`/`FSFRM` instructions. These instructions swap a value into the float CSRs by moving the source register to the CSR and the value in the CSR to the destination register. The Pcode implementation of this instruction attempted to do this without using a temporary, causing the value in the source register to be overwritten. This lack of a temporary when using aliased registers also occurs in the compressed instruction extension with the `C.JALR ra` instructions. This instruction should jump to the address held in the `ra` register and replace its value with the address of the following instruction, but was found to instead execute the instruction at the following address.

Both RISC-V and AArch64 exhibit bugs involving the architectural zero register, which should always return 0 and discard writes. The RISC-V SLEIGH specification tries to enforce this via a subtable that exports a temporary variable instead of the zero register itself. However, several instruction definitions directly reference the zero register, circumventing this mechanism and allowing non-zero values to be written when the register is used as a destination.

Additionally for RISC-V, the destination register for some instructions is intended to be 32 bits instead of the full register width. To handle this, a constructor `rdW` is used, which creates a temporary of the right size, sets its value to that of the destination register, and exports the temporary. When an instruction attempts to write to this aliased register, it would change the temporary but not the destination register. This prevented the `FCVT` instructions from outputting any result.

4.2.6 Unresolvable Issues

The problems we discussed previously can be addressed through careful editing of processor specifications. However, there are numerous issues requiring more fundamental design changes to simultaneously support accuracy of both decompilation and emulation use cases; we discuss these next.

I-11 **Unimplemented operations.** The SIMD and vector instructions across x86-64 and AArch64 are partially implemented in Pcode and partially with added custom Pcode ops, with the semantic section of RISC-V's vector extension completely unimplemented. A full Pcode implementation of these is possible, but would bloat the generated disassembly and output Pcode for static analysis tools.

I-12 **Floating point issues.** Floating point instructions contributed to a large number of discrepancies, as many architecture-specific aspects of the IEEE 754 standard are not exposed through the SLEIGH specification layer. SLEIGH does not enforce a rounding mode and instead uses the environment's default, while modern architectures (x86-64, AArch64, RISC-V) provide a means to change the default rounding mode, as well as override the rounding mode for individual instructions. Architectures with hardware support for floating point numbers generally include the common encodings of FP32 and FP64 which are supported in SLEIGH. However, x86-64 and AArch64 also support the FP16 and BF16 encoding which SLEIGH does not have. SLEIGH also chooses to propagate NaN payloads, while the IEEE 754 standard leaves this to the implementers' discretion. This differs from RISC-V and conditionally AArch64 which always return the canonical NaN. Properly fixing these discrepancies would require better floating point support from SLEIGH.

I-13 **Unpredictable behavior.** A set of discrepancies for x86-64, AArch64, and ARM/Thumb came from defined unpredictable outputs in related ISA specifications. But, these currently cannot be *fixed* as there is no single correct behavior.

I-14 **Memory alignment exceptions.** Many SIMD and atomic operations have stricter alignment requirements than standard load/store instructions. When an operand is provided at an insufficiently aligned address, an exception should be generated. However, there is no mechanism to annotate these requirements in a SLEIGH specification, meaning the emulator is unaware that an exception should occur. While extra Pcode could be used, this case is better handled by adding explicit alignment to memory accesses.

I-15 **Missing instruction semantics.** The x86-64 specification does not implement the auxiliary carry flag (`AF`) in *any* instruction, resulting in discrepancies in almost all arithmetic instructions. This is a known limitation of the specification (treated as a single bug) and likely remains unfixed because the auxiliary carry flag is rarely used in modern programs.

Importantly, based on our in-depth analysis of the issues uncovered, we detail possible solutions in recommended modifications to the SLEIGH DSL in Section 5.1.

4.2.7 Discrepancy-Bug Associations

Although the relationship between discrepancies and bugs is not a simple one-to-one mapping, Table 5 discusses how

Category	Unimpl.	Exec.	Flags	Float	PC	State
Decoding	5	11	0	0	0	9
Semantic Ordering	0	0	1	0	1	7
Incorrect Semantics	1	1	8	4	6	66
Aliasing	0	0	0	0	0	10

Table 5: Summary of the the root-cause bugs triaged and fixed and their association to observed discrepancies types.

triaged and fixed bugs in Table 4 relate to relevant observed discrepancies to distinguish those that relate to the bugs. Notably, a single bug may manifest in different types of discrepancies while a single discrepancy may be the result of multiple types of bugs. We observe up to three separate bugs triggered by a single discrepancy. Multiple discrepancies from the same bug are common, especially when incorrect semantics are encountered. Hence these numbers represent a lower bound. In addition, recall that *Alignment* related discrepancies are separated since these are generally unfixable without adding additional semantics to SLEIGH as we discuss in Section 5.1. To generate results, we used the related discrepancies leading to the identification of the bug during triaging to conduct the analysis. The associations in Table 5 demonstrate that systematic discrepancy detection serves as a strong indicator for underlying correctness problems. The distribution of bugs in Table 5 suggests *State* related issues should be prioritized in analysis. Although discrepancy classes are difficult to attribute directly to bugs, *State* and *Exception* related discrepancies are strong indicators of errors in the SLEIGH specification.

4.3 Effectiveness of Techniques

In this section we investigate the effectiveness of the designed technique to traverse a given specification to generate encoded instructions and discover specification bugs.

Instruction Generation. To quantify the contribution of individual components in GenSYS, we perform an ablation study over its constructor traversal and constraint-solving techniques. We focus on three key aspects to evaluate: i) *Simple constraints only* (ignoring all deferred constraints); ii) *Explicit constraints only* (solve deferred constraints, but excludes implied constraints from overlapping constructors);

Arch.	Simple Only		Explicit Only		No Backtracking	
	Δ *Acc* (↓)	*Missed*	Δ *Acc* (↓)	*Missed*	Δ *Acc* (↓)	*Missed*
x86-64	16.7%	995	12.3%	65	0%	0
AArch64	13.2%	98	13.7%	79	0%	11
ARM/Thumb	30.4%	302	15.9%	80	0%	75
RISC-V	57.7%	38	54.7%	38	0%	0
MSP430	69.6%	388	41.9%	58	0%	0

Table 6: Summary of instruction generation accuracy (based on constructors used for decoding compared to generation), and missed constructors compared to GenSys, when solving techniques are removed.

and iii) *No backtracking* (solve all constraints, but assume backtracking from subtables is impossible) versus GenSYS (solve all constraints with backtracking allowed).

We report two metrics to understand impact: i) Δ *Accuracy*—the decrease in generation accuracy relative to GenSYS, where accuracy is the percentage of generated instructions whose decode path matches the target constructors; and ii) *Missed*—the number of constructors for which no valid instruction encoding was found, despite a valid encoding being found by GenSYS.

Table 6 shows that simple constraint solving alone, *Simple Only*, significantly reduces accuracy, particularly when applied to the MSP430 and RISC-V SLEIGH specifications. Without translating simple constraints, many generated instructions match different constructors to those intended, resulting in missed constructors. Although, *Explicit constraints only*, shows a reduced accuracy drop, especially for MSP430, accuracy drops remain above 12% for each ISA. Then, with *No Backtracking* only, all of the generated instructions' decode path matches the target constructors, as with GenSYS, but leads to *Missed* constructors. Overall, disabling the constraint translation components significantly degrades test generation effectiveness across all evaluated ISAs, causing large regions of the specification to remain uncovered.

Comparison to naive approach. To further evaluate the overall effectiveness of the instructions and states generated by GenSYS, we can compare it to a naive approach devoid of our key techniques. Instead of modeling the SLEIGH decoding process, we simply select a constructor at random, set any bits that have fixed constraints for that constructor, and fill any remaining bits with random values. For each generated instruction, we randomize the input state to derive the instruction used by JuxtaPlayer for testing.

We compare the random approach to GenSYS on x86-64 across two metrics: i) *Constructors Hit*—the number of constructors for which at least one generated instruction correctly decodes to that constructor; and ii) *Bugs*—the number of bugs triggered from the 32 root-cause bugs discovered for x86-64. Notably, fully validating bug metrics would require a significant amount of manual analysis, as indeed performed in our primary analysis with InSPECtor, for the randomly generated instructions. Therefore, we automatically determine whether a bug was triggered. We manually annotated each bug with the conditions required for detecting it—a *canary* based on the constructors containing the bug. We provide canaries for each bug in our open-source GitHub repository.

The results of the naive approach are shown in Figure 4. After generating and testing 1.1×10^7 testcases taking 8 hours, the naive approach failed to: i) generate any instruction that matches 1249 constructors (38% miss); and ii) find 15 bugs (47% miss). To better illustrate the failures of the naive approach, we consider two of the missed bugs below:

`CVTSS2SI/CVTSD2SI`. These instructions are used to convert scalar floating-point values into integers. However,

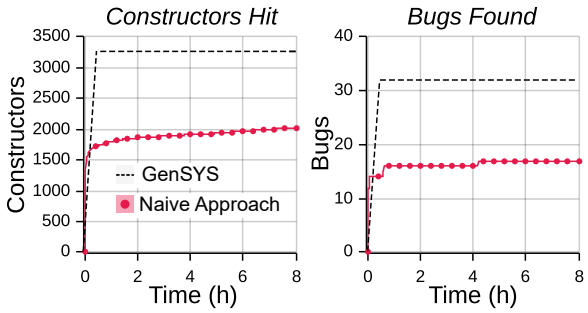


Figure 4: Constructors hit in the x86-64 SLEIGH specification and bugs found with the naive approach for testing with JuxtaPlayer. Notably, GenSYS generation and the execution of the resulting test cases takes <40 minutes for x86-64.

the SLEIGH specification contains a bug where the float-to-integer cast operation (`trunc`) is missing in the constructors that handle cases where the destination operand size matches the floating-point source operand size. These instructions have a complex encoding that requires a specific prefix byte and a multi-byte opcode, consequently they were never generated by the random approach. GenSYS generates constraints that force the appropriate prefix byte when generating this instruction and is able to find this bug.

PACKUSWB. This instruction converts 16-bit signed words (ranging from -2^{15} to $2^{15}-1$) in the source operand into 8-bit unsigned bytes (from 0 to 2^8-1) for the destination operand. Values outside the destination range are saturated (i.e., negative values become 0 and values greater than 255 become 255). However, the SLEIGH specification contains an off-by-one error in a comparison operation, causing an input of 255 to be converted to zero. Since this bug occurs only for one specific input value, random testing is highly unlikely to expose it. In contrast, GenSYS derives edge-case inputs from the SLEIGH semantics and generates targeted states around boundary conditions, allowing it to detect the bug.

5 Discussion

In addition to the limitations of this work and potential for future work, potential changes to the design of the SLEIGH specification language are discussed below. These suggested changes allow for improved maintainability, versatility, fidelity, and understandability of SLEIGH specifications.

5.1 Recommendations for Improving SLEIGH

The SLEIGH language is demonstrably effective for specifying instruction semantics, as evidenced by the more than 35 ISAs represented in SLEIGH within Ghidra. However, our study reveals that SLEIGH lacks features and constructs needed to express complex instruction behaviors suited for its applications in security workflows. Based on the issues uncovered in differential test result analysis in Section 4.2, we propose extensions to the SLEIGH language that could potentially lead to fewer bugs in specifications. We provide a summary of recommendations below and defer readers to Appendix C for an extended version.

General recommendations. Based on the issues we uncovered in Section 4.2.2-4.2.5, these recommendations aim to better support the key use cases for SLEIGH while reducing bugs during specification development and maintenance.

- **Add support for decompiler *hints*.** A challenge in writing SLEIGH specifications is balancing the need for high-fidelity semantics to facilitate emulation while producing clean decompilation output. Adding a mechanism for annotating instructions with decompiler hints would allow specification authors to provide guidance on how certain instructions should be represented in the decompiled output, without affecting the emulation semantics.
- **Introduce templated constructors and arrays.** Provide higher-level language constructs that reduce code duplication and copy-and-paste errors, while improving specification readability and maintainability.
- **Introduce improved accessor functions.** The current use of the `export` statement to return references from subtables is error-prone. Dedicated accessor functions would provide a clearer mechanism for exporting values and managing dependencies.
- **Support inline helper functions.** Inline helper functions would allow complex logic to be encapsulated in a reusable way, and could allow for more concise decompiler output if desired.

Recommendations to address the unresolvable issues. We identified problems, described in issues I-11–I-14, that are infeasible or impractical to resolve within the current SLEIGH language. To address these limitations, we propose the following extensions to the SLEIGH DSL:

- **Support SIMD extensions I-11.** Expressing SIMD semantics in Pcode is cumbersome because many repeated operations are performed on different parts of the input operands. Introduce vectorized Pcode operations or structured iteration constructs to represent SIMD instruction semantics more concisely.
- **More precise floating-point semantics I-12.** Introduce language constructs for better describing floating-point operations, including describing the floating point format (bfloat16 vs half-floats), rounding modes, and other related semantics.
- **Represent unpredictable and undefined values explicitly I-13.** Add mechanisms to model and isolate undefined or architecturally unpredictable ISA behavior more precisely.
- **Specify memory alignment requirements I-14.** Allow alignment constraints to be expressed explicitly when per-

forming memory operations.

5.2 Other Processor Specifications

InSPECtor is designed to address the challenges posed by the SLEIGH DSL in its extensive and diverse set of processor specifications but its core ideas—symbolically traversing a specification's decoding rules, extracting edge cases and state generation—can extend to other specification languages such as Sail or VADL. At a high level, languages such as VADL [15] and Sail [30] share structural similarities with SLEIGH: they describe instruction encodings alongside executable or interpretable semantics. VADL, as implemented in OpenVADL [15], provides a concise and formal representation of processor architectures, with explicit modeling of instruction formats and behavior. Similarly, Sail [30] offers a rich, formally grounded language for ISA semantics, often used in verification settings for architectures such as ARM. These commonalities suggest that our approach could generalize along two main dimensions. First, specification traversal—i.e., systematically enumerating decoding paths, instruction variants, or semantic rules—can be adapted to any language that exposes structured instruction definitions. Second, symbolic solving can be applied as long as the specification can be translated into a constraint representation. In principle, both VADL and Sail provide sufficient structure and formalism to support these operations.

However, adopting our approach requires solving challenges pertinent to the specification language, and traversal in each specification language is fundamentally different because each language exposes a different normalized representation and structure. In particular, more formally grounded specification languages such as Sail support richer decoding constructors (e.g., complex control flow), which can significantly increase solver complexity or introduce path explosion during specification traversal, requiring more aggressive pruning or summarization. Then, translating semantics into efficient encodings for a symbolic solver could be non-trivial, if the abstractions do not map directly to bit-precise constraints. Hence, the primary effort lies not in redesigning the symbolic methodology itself, but in translating language-specific representations into a model suitable for constraint solving and automated exploration.

5.3 False Positives

In general, discrepancies correspond to either bugs or limitations in the SLEIGH specification, thus defining a false positive is hard. Each discrepancy identified represents a possible bug, but cannot be easily deduplicated without manual effort, so we report discrepancies, as done in related works in CPU emulator testing [23–25]. Then, we discuss bugs in the context of root causes fixed, not the number of buggy instructions, a more inflated number.

The large number of discrepancies found for many of the identified bugs is a major contributor to the observed difference between the bug and discrepancy counts. In addition to discrepancies mapped to root-causes, other contributors include ISA-defined unpredictable behaviors, unimplemented semantics, or issues which cannot be fixed without extending SLEIGH, discussed in Section 4.2.6. Due to limitations of the SLEIGH language, these issues do not represent an error in the specification, and consequently could be considered false positives; however, our recommendations for improving SLEIGH, discussed in Section 5.1, would allow addressing the sources of these discrepancies while helping to improve specification veracity.

5.4 Limitations & Future Work

Despite our extensive efforts, our work is not without limitations. Although we sought to test the correctness of a specification, our approach will not determine the completeness of a specification, since instructions missing from a SLEIGH specification are not generated, and therefore not tested. However, these instructions exhibit a well-defined failure mode: whenever an unknown instruction encoding is encountered, an invalid instruction exception will be generated in, for instance, SLEIGH-based emulators. This behavior typically makes any missed instructions easy to detect in real-world targets. But, in disassemblers, missing instructions in specifications can impact control-flow recovery where invalid instructions are used as heuristics for identifying unreachable or invalid code regions. In these cases, missing instructions may cause the analysis to misattribute control-flow errors, making it harder to localize the root cause. Future work should consider not only correctness but also achieving completeness as an attribute of specification testing.

Some ISAs include instructions that alter the execution of subsequent code. For example, to allow for a more compact instruction encoding, ARM/Thumb includes an If-Then-Else (`ITE`) instruction in THUMB mode that changes up to four consecutive instructions into variants that are conditionally executed based on the value of the status register. Since the approach presented here generates and tests a single instruction at a time, it does not test these cases. Therefore, future work can consider advancing the generation process to identify and generate the necessary instruction sequences to test these variants in the specification.

6 Related Work

We acknowledge the significant efforts devoted in past studies to test CPU emulators. EmuFuzzer [25] tested x86-64 CPU emulators using a hardware reference to detect structures within the instruction by observing when mutating a particular byte causes an exception. KEmuFuzz [24] extends EmuFuzzer by adding support for fuzzing virtual machines

and hypervisors through carefully crafted templates that include snippets of both user- and system-level instructions, but retained the same instruction generation method as EmuFuzzer. The approach is suited for testing x86-64 emulators running in protected mode (32-bit instructions), requires filtering branch instructions due to execution environment limitations, and only identifies imprecise byte-level structure in instructions. In contrast, PokeEMU [23] generates test instructions by symbolically executing the emulator's binary instruction decoding logic. This is similar to the constructor traversal process adopted in GenSYS, but applied to the emulator binary. Fast PokeEMU [37] builds on PokeEMU by reducing test case execution overhead by merging multiple instructions into a single test case. In contrast to InSPECtor, PokeEMU does not generate edge case-triggering input states, as the value of input operands is not used in the decoding logic. Further, InSPECtor's objective to guarantee complete constructor coverage implies that the method devised can find bugs in unreachable constructors.

Other testing strategies are found in studies examining binary lifters. MeanDiff [20] compares binary lifters by translating each intermediate representation (IR) into a common IR. Once translated, an SMT solver is employed to check for symbolic equivalence. However, due to the complexity of the translation process, certain complex instructions are omitted from analysis. In contrast, JuxtaPlayer, akin to CPU-emulator testing methods, uses a hardware oracle as a reference and determines equivalence by testing selected initial input states, avoiding the need to check the symbolic equivalence of semantically complex instructions, such as floating-point operations. Notably, a prior study analyzed the limitation of Pcode's formal semantics [26]. This led to several recommendations to improve the Pcode definitions, documentation and resolve bugs in Ghidra. This complements the recommendations resulting from our extensive work across five ISAs to improve SLEIGH and the processor specifications in Section 5.1.

Alongside these studies, we also acknowledge studies developing techniques to uncover flaws in hardware-assisted virtual machines [3, 14] or methods to uncover discrepancies in the textual representation of instructions generated by disassemblers [28]. Other approaches aim to detect unexpected changes in the processor state of individual hardware CPUs arising from undocumented instructions [11], manufacturing defects [31], or hardware implementation issues [31].

In general, similar to our study, these works across several domains also aim to identify discrepancies. Notably, discrepancies, even one known to differ between hardware implementations, can be exploited by malware for emulator or VM detection, as demonstrated in previous studies [29, 32, 33].

7 Conclusion

This paper introduced a methodology for generating instructions and initial input states that fully explore all machine code sequences decodable by SLEIGH processor specifications. This led to the development of InSPECtor with GenSYS and JuxtaPlayer. GenSYS is an instruction and state generator used to produce over 1,200,000 test cases across 5 different ISAs, covering all instructions and addressing modes, as well as edge cases. The test cases were used to evaluate the correctness of specifications using a differential testing strategy, realized in JuxtaPlayer. The process revealed systematic issues in 5 SLEIGH specifications and highlighted areas where improvements to the SLEIGH specification language could reduce future defects. Improving the veracity of SLEIGH specifications enhances the reliability and confidence in downstream tools that depend on their correctness. Importantly, our testing strategy and tools support automated correctness evaluation of SLEIGH specifications into the future.

Acknowledgements

The work was supported by the Next Generation Technology Fund (NGTF)—administered by Commonwealth Scientific and Industrial Research Organisation (CSIRO) and Defence Science and Technology Group (DSTG) in Australia, The Advanced Strategic Capabilities Accelerator (ASCA) program and DSTG, Australia. The authors would like to thank and acknowledge Om Mandavia for the preliminary investigations.

A Ethical Considerations

Our work presents a method and tool design for testing and evaluating processor specifications focusing on the open source SLEIGH specifications. Our goal is to improve the truthfulness of the tools used by the security community, such as Ghidra and CPU emulators, that rely on the truthfulness of the SLEIGH specifications, and thereby helping to improve the security of software systems and the safety and security of end-users of those systems.

Through our analysis with InSPECtor, we identified a substantial number of defects in existing SLEIGH specifications. While documenting these issues is essential for improving the correctness and reliability of disassembly and analysis frameworks, public discussion of such defects carries some dual-use risks. In particular, an adversarial actor could potentially misuse specification inconsistencies to craft binaries that evade static or dynamic analysis, mislead emulators, or otherwise undermine defensive tooling.

To mitigate these risks, we followed established responsible-disclosure practices. All defects discovered during this work were communicated directly to the Ghidra project maintainers, together with the corresponding fixes that we validated internally. Our intention is to strengthen the ecosystem of analysis tools rather than expose previously unknown issues without remediation. We emphasize that any future defects uncovered through the use of this tool should

likewise be discussed and disclosed responsibly to the maintainers of the Ghidra project following the guidelines therein.

Despite these concerns, we believe the benefits of improving specification quality, strengthening capabilities and correctness of widely used analysis tools, employing those specifications, used by security practitioners and improving transparency outweigh the risks or potential for misuse. Ultimately, our commitment to responsible disclosure and to support the maintainers of widely used tooling guided the design of this work and will continue to guide its future development.

B Open Science

InSPECtor code and artifacts in the study are publicly archived at https://doi.org/10.6084/m9.figshare.32507811.v1, including.

- **GenSYS**, the main implementation of our techniques, used to traverse the SLEIGH specifications and generate the test cases summarized in Table 2.
- **JuxtaPlayer**, used for executing the test cases on both real-world hardware and in SLEIGH emulators. As well as for for identifying and grouping the discrepancies for Table 3.
- **Documentation**, on how to use each tool in the form of multiple `README` files, and on how to recreate the ablation study in Section 4.3.
- **Bug Fixes**, individualized in our fork of the Ghidra project while they undergo the review process.

We open-source the InSPECtor project on GitHub at https://github.com/Sleigh-InSPECtor.

References

[1] Intel® 64 and ia-32 architectures software developer's manual. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html, October 2025.

[2] Open SSL Github. https://github.com/openssl/openssl/blob/afd8e29c360376420ea676581aa5d50b6027d069/crypto/bn/asm/x86_64-mont.pl#L1168-L1169, 2025. Access 26 March 2026.

[3] N. Amit, D. Tsafrir, A. Schuster, A. Ayoub, and E. Shlomo. Virtual cpu validation. In *Symposium on Operating Systems Principles (SOSP)*, pages 311–327, 2015.

[4] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K. E. Gray, R. M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell. ISA semantics for armv8-a, RISC-v, and CHERI-MIPS. In *ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 2019.

[5] N. Bjørner, A.-D. Phan, and L. Fleckenstein. νZ-an optimizing SMT solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 194–199, 2015.

[6] J. Blackthorne, A. Bulazel, A. Fasano, P. Biernat, and B. Yener. Avleak: Fingerprinting antivirus emulators through black-box testing. In *USENIX Workshop on Offensive Technologies (WOOT)*, 2016.

[7] Z. Chen, S. L. Thomas, and F. D. Garcia. Metaemu: An architecture agnostic rehosting framework for automotive firmware. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 515–529, 2022.

[8] M. Chesser, S. Nepal, and D. C. Ranasinghe. Icicle: A re-designed emulator for grey-box firmware fuzzing. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023.

[9] S. Dasgupta, S. Dinesh, D. Venkatesh, V. S. Adve, and C. W. Fletcher. Scalable validation of binary lifters. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, page 655–671, 2020.

[10] F. Dola and G. Orlando. Ghidralligator: Emulate and fuzz the embedded world. https://www.cyber.airbus.com/ghidralligator_emulate_and_fuzz_the_embedded_world/, 2023.

[11] C. Domas. Breaking the x86 ISA. *Black Hat*, 2017.

[12] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++: Combining incremental steps of fuzzing research. In *USENIX Workshop on Offensive Technologies (WOOT)*, 2020.

[13] A. Fog. The microarchitecture of intel, amd and via cpus: An optimization guide for assembly programmers and compiler makers. https://www.agner.org/optimize/microarchitecture.pdf, 2023.

[14] P. Fonseca, X. Wang, and A. Krishnamurthy. MultiNyx: a multi-level abstraction framework for systematic analysis of hypervisors. In *European Conference on Computer Systems (EuroSys)*, 2018.

[15] F. Freitag, L. Halder, B. Huber, B. Kasper, M. Nestler, K. Per, M. Raschhofer, A. Ripar, J. Zottele, and A. Krall. Openvadl: An open source implementation of the vienna architecture description language. In *Architecture of Computing Systems (ARCS)*, pages 156–171, 2026.

[16] R. Gopinath, C. Jensen, and A. Groce. Code coverage for suite evaluation by developers. In *International conference on software engineering (ICSE)*, pages 72–82, 2014.

[17] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee. Difuzzrtl: Differential fuzz testing to find CPU bugs. In *IEEE Symposium on Security and Privacy (SP)*, 2021.

[18] M. Jiang, T. Xu, Y. Zhou, Y. Hu, M. Zhong, L. Wu, X. Luo, and K. Ren. EXAMINER: Automatically locating inconsistent instructions between real devices and CPU emulators for ARM. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 846–858, 2022.

[19] S. Kim, M. Faerevaag, M. Jung, S. Jung, D. Oh, J. Lee, and S. K. Cha. Testing intermediate representations for binary analysis. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, page 353–364, 2017.

[20] S. Kim, M. Faerevaag, M. Jung, S. Jung, D. Oh, J. Lee, and S. K. Cha. Testing intermediate representations for binary analysis. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 353–364, 2017.

[21] P. S. Kochhar, F. Thung, and D. Lo. Code coverage and test suite effectiveness: Empirical study with real bugs in large systems. In *IEEE international conference on software analysis, evolution, and reengineering (SANER)*, pages 560–564, 2015.

[22] a. L. C. Kudu Dynamics, LLC. Styx emulator. https://github.com/styx-emulator/styx-emulator, 2025.

[23] L. Martignoni, S. McCamant, P. Poosankam, D. Song, and P. Maniatis. Path-exploration lifting: Hi-Fi tests for Lo-Fi emulators. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 337–348, 2012.

[24] L. Martignoni, R. Paleari, G. Fresi Roglia, and D. Bruschi. ACM SIGSOF testing system virtual machines. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 171–182, 2010.

[25] L. Martignoni, R. Paleari, G. F. Roglia, and D. Bruschi. Testing CPU emulators. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 261–272, 2009.

[26] N. Naus, F. Verbeek, D. Walker, and B. Ravindran. A formal semantics for p-code. In *Verified Software. Theories, Tools and Experiments (VSTTE)*, page 111–128, 2022.

[27] T. Ormandy. Reptar. https://lock.cmpxchg8b.com/reptar.html, 2023.

[28] R. Paleari, L. Martignoni, G. Fresi Roglia, and D. Bruschi. N-Version disassembly: Differential testing of x86 disassemblers. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 265–274, 2010.

[29] R. Paleari, L. Martignoni, G. F. Roglia, and D. Bruschi. A fistful of red-pills: How to automatically generate procedures to detect cpu emulators. In *USENIX Workshop on Offensive Technologies (WOOT)*, volume 41, page 86, 2009.

[30] A. Reid. Trustworthy specifications of ARM® v8-A and v8-M system level architecture. In *Formal Methods in Computer-Aided Design (FMCAD)*, pages 161–168, 2016.

[31] K. Serebryany, M. Lifantsev, K. Shtoyk, D. Kwan, and P. Hochschild. SiliFuzz: Fuzzing CPUs by proxy. *arXiv preprint arXiv:2110.11519*, 2021.

[32] H. Shi, A. Alwabel, and J. Mirkovic. Cardinal pill testing of system virtual machines. In *USENIX Security Symposium (USENIX Security)*, pages 271–285, 2014.

[33] H. Shi, J. Mirkovic, and A. Alwabel. Handling anti-virtual machine techniques in malicious software. *ACM Transactions on Privacy and Security (TOPS)*, 21(1):1–31, 2017.

[34] F. Solt, K. Ceesay-Seitz, and K. Razavi. Cascade: CPU fuzzing via intricate program generation. In *USENIX Security Symposium (USENIX Security)*, pages 1–18, 2024.

[35] The National Security Agency (NSA). Ghidra: Software reverse engineering framework. https://ghidra-sre.org/, 2019.

[36] The National Security Agency (NSA). SLEIGH: A language for rapid processor specification. https://ghidra.re/ghidra_docs/languages/html/sleigh.html, 2023.

[37] Q. Yan and S. McCamant. Fast PokeEMU: Scaling generated instruction tests using aggregation and state chaining. In *ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*, pages 71–83, 2018.

C Additional Material

An **extended version** of the paper, including detailed bug tables, with additional material and the open-source InSPECtor project is available on GitHub at https://github.com/Sleigh-InSPECtor.